

Appendix

A PROMPT USED FOR BASELINE APPROACH: FRONTIER MODELS TO GENERATE SYNTHETIC (NL, SQL) PAIRS

Below is the exact prompt used by the baseline approach that uses a frontier model (Claude Opus 4.7 model) to generate data synthetically. The two parts of this prompt are a system prompt and a user prompt are outlined below, along with the RAG blocks filled per-database at runtime.

A.1 System Prompt

You are a senior data engineer building a HARD text-to-SQL evaluation benchmark on top of real SQLite databases. Your job is to invent challenging, realistic analytical questions and the single correct SQL query that answers each one.

Follow every rule exactly:

1. **DIFFICULTY.** Each SQL query MUST join at least 5 DISTINCT base tables (ideally 5-7). Increase difficulty with aggregation (COUNT / SUM / AVG / MIN / MAX), GROUP BY, HAVING, multi-condition WHERE filters, or ordering with LIMIT. Prefer questions a strong model would find genuinely hard.

2. **CORRECTNESS & EXECUTABILITY.** The SQL must be a SINGLE valid SQLite SELECT statement that runs without error and returns at least one row. Use ONLY tables and columns that appear in the provided schema. Join tables ONLY through the provided foreign-key relationships. Quote identifiers that contain spaces or special characters with double quotes.

3. **GROUNDING LITERALS.** Any value used in a WHERE/HAVING clause must be a real value of that column. Prefer the sampled column values provided; never invent values that would return zero rows.

4. **FAITHFUL QUESTIONS.** The natural-language question must be unambiguous and fully answerable by executing the SQL against the database - no external knowledge, no information the query does not actually compute. Do not leak SQL syntax into the question.

5. **NO HALLUCINATION.** Never reference a table or column that is not in the schema. If you are unsure a column exists, do not use it.

OUTPUT CONTRACT. Return ONLY a JSON array (no prose, no markdown fences). Each element is an object with exactly these keys: - "question": string, the natural-language question. - "sql": string, the single SQLite SELECT that answers it. - "join_path": array of the table names joined, in order. - "focus_columns": array of the key "table.column" strings the question is about. - "aggregation": string, e.g. "COUNT(*)", "AVG(...)", "GROUP BY ...", or "" if none. - "rationale": string, one sentence on why this question is hard.

A.2 User Prompt

TARGET DATABASE:

db_id

=== SCHEMA ===
schema_card

=== FOREIGN-KEY JOIN CONDITIONS ===

fk_block

=== CANDIDATE MULTI-TABLE JOIN CHAINS (>= 5 tables; use or ignore) ===

hint_block

=== SAMPLED REAL COLUMN VALUES (use these for grounded WHERE literals) ===

value_block

=== RETRIEVED GOLD EXAMPLES FROM THIS DATABASE (for style and SQL patterns) ===

exemplar_block

=== TASK ===

Generate 6 NEW, DISTINCT, HARD (question, SQL) data points for the 'db_id' database. Each SQL must join at least 5 distinct tables (target 5-7) and must execute on SQLite returning >= 1 row. Make the questions diverse (different tables, filters, and aggregations) and do not copy the retrieved examples verbatim.

Return ONLY the JSON array described in the system prompt.

A.3 Description of the Baseline Prompt

What fills the RAG placeholders (per database): (i) schema_card - the full schema: every table with its columns/types; (ii) fk_block: the database's foreign-key join conditions (the only joins the model is told it may use); (iii) hint_block: up to 4 candidate at least 5-table FK join chains (n_pat_hints=4), offered as optional scaffolding; (iv) value_block - sampled real column values: up to 5 distinct values from up to 40 text columns (value_sample_vals=5, value_sample_cols=40), for grounded WHERE literals; (v) exemplar_block: 4 retrieved gold (NL, SQL) exemplars from that same database (n_exemplars=4), for style/pattern.

A.4 A Fully Materialized User Prompt (from baseline prompt) for BIRD Validation Database (e.g., financial)

----- SYSTEM PROMPT -----

You are a senior data engineer building a HARD text-to-SQL evaluation benchmark on top of real SQLite databases. Your job is to invent challenging, realistic analytical questions and the single correct SQL query that answers each one.

Follow every rule exactly:

1. **DIFFICULTY.** Each SQL query MUST join at least 5 DISTINCT base tables (ideally 5-7). Increase difficulty with aggregation (COUNT / SUM / AVG / MIN / MAX), GROUP BY, HAVING, multi-condition WHERE filters, or ordering with LIMIT. Prefer questions a strong model would find genuinely hard.

2. **CORRECTNESS & EXECUTABILITY.** The SQL must be a SINGLE valid SQLite SELECT statement that runs without error and returns at least one row. Use ONLY tables and columns that appear

in the provided schema. Join tables ONLY through the provided foreign-key relationships. Quote identifiers that contain spaces or special characters with double quotes.

3. GROUNDED LITERALS. Any value used in a WHERE/HAVING clause must be a real value of that column. Prefer the sampled column values provided; never invent values that would return zero rows.

4. FAITHFUL QUESTIONS. The natural-language question must be unambiguous and fully answerable by executing the SQL against the database - no external knowledge, no information the query does not actually compute. Do not leak SQL syntax into the question.

5. NO HALLUCINATION. Never reference a table or column that is not in the schema. If you are unsure a column exists, do not use it.

OUTPUT CONTRACT. Return ONLY a JSON array (no prose, no markdown fences). Each element is an object with exactly these keys: - "question": string, the natural-language question. - "sql": string, the single SQLite SELECT that answers it. - "join_path": array of the table names joined, in order. - "focus_columns": array of the key "table.column" strings the question is about. - "aggregation": string, e.g. "COUNT(*)", "AVG(...)", "GROUP BY ...", or "" if none. - "rationale": string, one sentence on why this question is hard.

----- USER PROMPT -----

TARGET DATABASE: financial

=== SCHEMA ===

Database: financial

Table account (account_id integer PRIMARY KEY, district_id integer, frequency text, date date)

Table card (card_id integer PRIMARY KEY, disp_id integer, type text, issued date)

Table client (client_id integer PRIMARY KEY, gender text, birth_date date, district_id integer)

Table disp (disp_id integer PRIMARY KEY, client_id integer, account_id integer, type text)

Table district (district_id integer PRIMARY KEY, A2 text, A3 text, A4 text, A5 text, A6 text, A7 text, A8 integer, A9 integer, A10 real, A11 integer, A12 real, A13 real, A14 integer, A15 integer, A16 integer)

Table loan (loan_id integer PRIMARY KEY, account_id integer, date date, amount integer, duration integer, payments real, status text)

Table order (order_id integer PRIMARY KEY, account_id integer, bank_to text, account_to integer, amount real, k_symbol text)

Table trans (trans_id integer PRIMARY KEY, account_id integer, date date, type text, operation text, amount integer, balance integer, k_symbol text, bank text, account integer)

=== FOREIGN-KEY JOIN CONDITIONS (join only through these)

===

account.district_id = district.district_id

card.disp_id = disp.disp_id

client.district_id = district.district_id
disp.client_id = client.client_id
disp.account_id = account.account_id
loan.account_id = account.account_id
order.account_id = account.account_id
trans.account_id = account.account_id

=== CANDIDATE MULTI-TABLE JOIN CHAINS (>= 5 tables; use or ignore) ===

hint 1: loan -> account -> district -> client -> disp -> card hint 2: order -> account -> district -> client -> disp -> card hint 3: trans -> account -> district -> client -> disp -> card hint 4: account -> district -> client -> disp -> card

=== SAMPLED REAL COLUMN VALUES (use these for grounded WHERE literals) === account.frequency: 'POPLATEK MESICNE', 'POPLATEK TYDNE', 'POPLATEK PO OBRATU' card.type: 'gold', 'classic', 'junior' client.gender: 'F', 'M' disp.type: 'OWNER', 'DISPONENT' district.A2: 'Hl.m. Praha', 'Benesov', 'Beroun', 'Kladno', 'Kolin' district.A3: 'Prague', 'central Bohemia', 'south Bohemia', 'west Bohemia', 'north Bohemia' district.A4: '1204953', '88884', '75232', '149893', '95616' district.A5: '0', '80', '55', '63', '65' district.A6: '0', '26', '29', '30', '23' district.A7: '0', '6', '4', '1', '7' loan.status: 'A', 'B', 'D', 'C' order.bank_to: 'YZ', 'ST', 'QR', 'WX', 'CD' order.k_symbol: 'SIPO', 'UVER', 'POJISTNE', 'LEASING' trans.type: 'PRIJEM', 'VYDAJ', 'VYBER' trans.operation: 'VKLAD', 'PREVOD Z UCTU', 'PREVOD NA UCET', 'VYBER', 'VYBER KARTOU' trans.k_symbol: 'SIPO', 'SLUZBY', 'UVER', 'POJISTNE', 'DUCHOD' trans.bank: 'AB', 'YZ', 'ST', 'QR', 'WX'

=== RETRIEVED GOLD EXAMPLES FROM THIS DATABASE (for style and SQL patterns) === Example 1 (BIRD difficulty: moderate):

Question: Provide the IDs and age of the client with high level credit card, which is eligible for loans. Evidence: the credit card is high-level refers to card.type = 'gold'; eligible for loans refers to disp.type = 'OWNER' SQL: SELECT T1.client_id, STRFTIME('%Y', CURRENT_TIMESTAMP) - STRFTIME('%Y', T3.birth_date) FROM disp AS T1 INNER JOIN card AS T2 ON T2.disp_id = T1.disp_id INNER JOIN client AS T3 ON T1.client_id = T3.client_id WHERE T2.type = 'gold' AND T1.type = 'OWNER'

Example 2 (BIRD difficulty: challenging): Question: For the client whose loan was approved first in 1993/7/5, what is the increase rate of his/her account balance from 1993/3/22 to 1998/12/27? Evidence: Increase rate of his/her account balance = [(balance of date A - balance of date B) / balance of Date B] * 100 SQL: SELECT CAST((SUM(IIF(T3.date = '1998-12-27', T3.balance, 0)) - SUM(IIF(T3.date = '1993-03-22', T3.balance, 0))) AS REAL) * 100 / SUM(IIF(T3.date = '1993-03-22', T3.balance, 0)) FROM loan AS T1 INNER JOIN account AS T2 ON T1.account_id = T2.account_id INNER JOIN trans AS T3 ON T3.account_id = T2.account_id WHERE T1.date = '1993-07-05'

Example 3 (BIRD difficulty: challenging): Question: Please list the account types that are not eligible for loans, and the average income of residents in the district where the account is located exceeds 8000butisnomorethan9000. Evidence: A11 represents the average salary; Salary and income share the similar meanings; when the account type = 'OWNER', it's eligible for loans SQL: SELECT T3.type FROM district AS T1 INNER JOIN account AS

T2 ON T1.district_id = T2.district_id INNER JOIN disp AS T3 ON T2.account_id = T3.account_id WHERE T3.type != 'OWNER' AND T1.A11 BETWEEN 8000 AND 9000

Example 4 (BIRD difficulty: challenging): Question: List out the account numbers of female clients who are oldest and has lowest average salary, calculate the gap between this lowest average salary with the highest average salary? Evidence: Female means gender = 'F'; A11 refers to average salary; Gap = highest average salary - lowest average salary; If the person A's birthdate > B's birthdate, it means that person B is older than person A. SQL: SELECT T1.account_id , (SELECT MAX(A11) - MIN(A11) FROM district) FROM account AS T1 INNER JOIN district AS T2 ON T1.district_id = T2.district_id INNER JOIN disp AS T3 ON T1.account_id = T3.account_id INNER JOIN client AS T4 ON T3.client_id = T4.client_id WHERE T2.district_id = (SELECT district_id FROM client WHERE gender = 'F' ORDER BY birth_date ASC LIMIT 1) ORDER BY T2.A11 DESC LIMIT 1

=== TASK ===

Generate 6 NEW, DISTINCT, HARD (question, SQL) data points for the 'financial' database. Each SQL must join at least 5 distinct tables (target 5-7) and must execute on SQLite returning ≥ 1 row. Make the questions diverse (different tables, filters, and aggregations) and do not copy the retrieved examples verbatim. Return ONLY the JSON array described in the system prompt.

B ERROR ANALYSIS ON THE BIRD DEVELOPMENT SET

We analyze the predictions of the *single-model* text-to-SQL pipeline on the BIRD development set. The variant studied is the phase-3 configuration with **contrastive exemplars** (proxy-bank retrieval), using *claude-opus-4.8* as the generator (with 5 contrastive exemplars using our SPLICE generated 750 challenging <NLQ, SQL> pairs data). Accuracy is measured by *execution match* (EX): a prediction is correct iff its result set equals that of the gold query (order-sensitive only when the gold query is ordered).

We then perform an error study over every failing prediction. The accuracy verdict is taken from the objective execution signal, whereas the error *categorization* is produced by an LLM-as-a-Judge (*claude-opus-4.8*, used directly with no external proxy) that contrasts each predicted query against the gold query, the NL question, the associated evidence, and the database schema. Each error is assigned exactly one top-level class (*structural* vs. *syntactic*) and, if structural, exactly one sub-category.

B.1 Overall accuracy

Table 1 summarizes execution accuracy on the 1,534-question dev set. The pipeline solves 1,083 questions (70.6%) and fails on 451, which form the population for the error study.

B.2 Structural vs. syntactic errors

We split the 451 failures by whether the predicted SQL is executable. A *syntactic* (execution-failure) error is one whose query is malformed or otherwise fails to produce a result (parse error, non-SQL output, or timeout); a *structural* error is a well-formed query that executes but returns the wrong result set. As Table 2 shows, structural

Table 1: Execution accuracy (EX) of the single-model + contrastive pipeline on BIRD dev, overall and by difficulty.

Difficulty	#Questions	#Correct	#Errors	EX (%)
Simple	925	699	226	75.6
Moderate	464	293	171	63.1
Challenging	145	91	54	62.8
All	1534	1083	451	70.6

Table 2: Top-level error distribution over the 451 BIRD-dev failures.

Error class	Count	% of errors
Structural (executes, wrong result)	449	99.6
Syntactic / execution-failure	2	0.4
Total	451	100.0

Table 3: Definitions of the structural error sub-categories.

Sub-category	Definition
Wrong projection / output columns	Filtering/joins are right, but the SELECT returns the wrong, extra, or missing columns.
Wrong aggregation / filtering	Wrong or missing aggregate (COUNT/SUM/AVG/MIN/MAX), GROUP BY, or WHERE/HAVING condition.
Wrong join-path	Correct tables joined via wrong keys / foreign-key path, or a join type (INNER/LEFT) that changes the result.
DISTINCT / duplicate handling	Missing (or spurious) DISTINCT or duplicate-row handling that changes the row multiplicity.
Look-alike column	Uses a wrong but similarly named / plausible column in place of the required one.
Wrong value / literal	Correct column but wrong constant, string, case, spelling, or unit in a filter.
Ordering / LIMIT	Wrong or missing ORDER BY key/direction or LIMIT/top-N/tie handling.
Other semantic	Genuinely wrong logic not covered above.
Omitted required table	A table/join needed to reach the answer is missing (under-joining).
Unnecessary join	An extra, unneeded table/join causes wrong rows, duplicates, or over-counting.

errors overwhelmingly dominate (99.6%): the model almost never emits invalid SQL, and virtually all failures are *semantic* mismatches with the intended query. Only two syntactic cases occur—one non-SQL output that raised a syntax error, and one valid-but-timing-out query. Figure 2 visualizes this split.

B.3 Taxonomy of structural errors

Within the 449 structural errors we distinguish ten sub-categories capturing where the query diverges from the gold logic. Table 3 defines them, and Table 4 reports their distribution; Figure 1 plots the same counts.

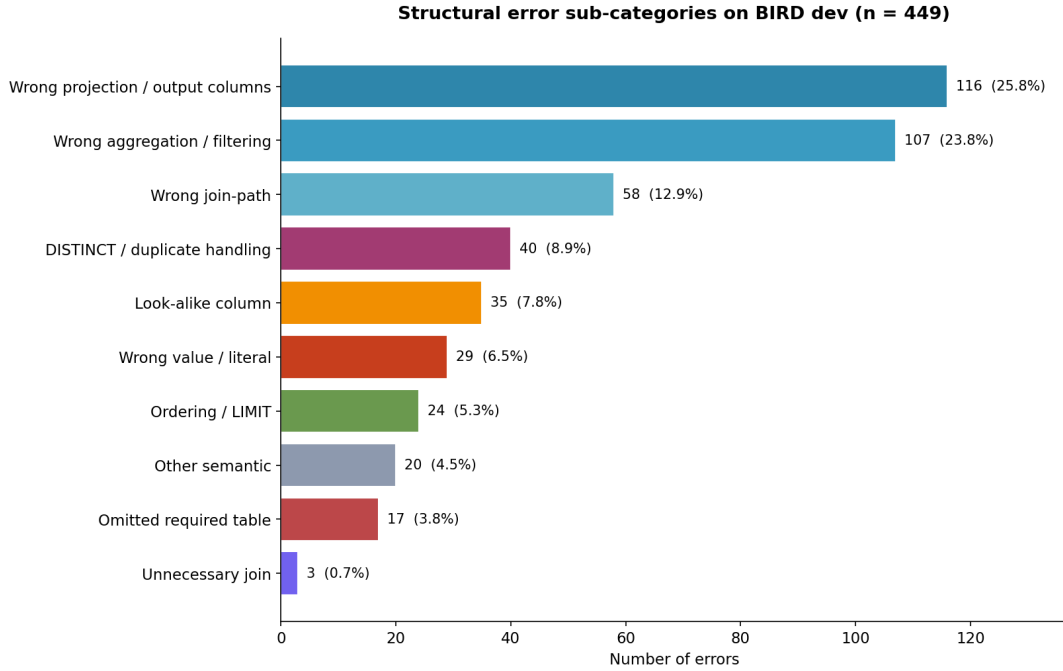


Figure 1: Structural error sub-categories on BIRD dev ($n=449$).

BIRD dev errors: Structural vs Syntactic
($n = 451$ execution failures of 1534)

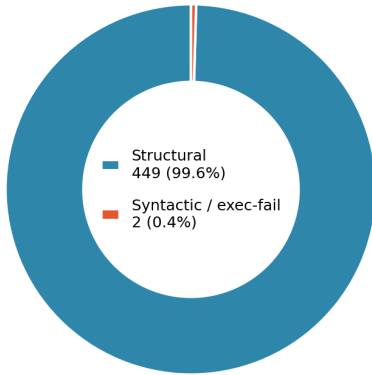


Figure 2: Structural vs. syntactic error split on BIRD dev ($n=451$ execution failures out of 1,534).

B.4 Discussion

Three sub-categories account for nearly two-thirds of all structural errors: *wrong projection/output columns* (25.8%), *wrong aggregation/filtering* (23.8%), and *wrong join-path* (12.9%). Together with *look-alike column* selection (7.8%), these point to *schema-linking* and *output-shaping* as the dominant failure modes rather than SQL well-formedness. Errors of row-multiplicity (*DISTINCT/duplicates*,

Table 4: Distribution of structural error sub-categories ($n=449$). Percentages are of structural errors.

Structural sub-category	Count	% of struct.
Wrong projection / output columns	116	25.8
Wrong aggregation / filtering	107	23.8
Wrong join-path	58	12.9
DISTINCT / duplicate handling	40	8.9
Look-alike column	35	7.8
Wrong value / literal	29	6.5
Ordering / LIMIT	24	5.3
Other semantic	20	4.5
Omitted required table	17	3.8
Unnecessary join	3	0.7
Total	449	100.0

8.9%) and *ordering/LIMIT* (5.3%) form a second tier tied to BIRD’s frequent top- N and counting questions. Conversely, connectivity errors are rare: *omitted required table* and *unnecessary join* together are under 5%, indicating the contrastive exemplars help the model recover the correct set of tables even when it mishandles the projected columns or predicates.

C STRUCTURAL COVERAGE ANALYSIS ON BIRD DEVELOPMENT SET

In this section, we provide a detailed structural coverage analysis on BIRD development set.

C.1 BIRD Development Schema: Formula 1

In the BIRD FORMULA 1 development database, the 174 development SQLs cover only 34 of the 1374 enumerated simple schema paths, corresponding to just 2.5% overall coverage. Coverage decreases dramatically with path depth: the development set covers 89.5% of length-1 paths, but only 22.7% of length-2 paths and 1.5% of length-3 paths, while covering none of the 294 length-4 paths, 326 length-5 paths, 444 length-6 paths, or 92 length-7 paths. Thus, despite a substantial number of development SQLs, the enormous combinatorial path space induced by the 13-table schema remains almost entirely uncovered, with complete absence of direct supervision for every schema path requiring four or more foreign-key traversals.

C.2 BIRD Development Schema: Superhero

In the BIRD SUPERHERO development database, the 129 development SQLs cover 36 of the 45 enumerated simple schema paths, corresponding to 80.0% overall coverage. Coverage is complete for length-1 paths and remains high for length-2 paths at 91.3%, but decreases to 50% for length-3 paths and 0% for the single length-4 path (attribute $\rightarrow$ hero_attribute $\rightarrow$ superhero $\leftarrow$ hero_power $\leftarrow$ superpower). Thus, even though the development set covers most of the schema-path universe overall, the only maximum-depth relational structure remains completely unobserved, illustrating that high aggregate coverage can still coexist with a blind spot at the deepest schema region.

C.3 BIRD Development Schema: Student Club

In the BIRD STUDENT CLUB development database, the 158 development SQLs cover 20 of the 50 enumerated simple schema paths, corresponding to 40.0% overall coverage. Coverage decreases progressively with path depth: the development set covers 100% of length-1 paths, 64.3% of length-2 paths, and only 27.3% of length-3 paths, while covering none of the 11 length-4 paths or 6 length-5 paths. Thus, although every individual foreign-key relationship is represented in the development set, coverage becomes increasingly sparse for multi-hop structures, leaving the deepest regions of the schema entirely without direct supervision.

C.4 BIRD Development Set: European Football 2

In the BIRD EUROPEAN FOOTBALL 2 development database, the 129 development SQLs cover 6 of the 11 enumerated simple schema paths, corresponding to 54.5% overall coverage. Coverage is complete for length-1 paths but drops to 33.3% for length-2 paths and to 0% for both length-3 and length-4 paths. Thus, while every individual foreign-key relationship is exercised somewhere in the development set, the data provides no direct supervision for any schema structure requiring three or more consecutive foreign-key traversals.

C.5 BIRD Development Set: Codebase Community

In the BIRD CODEBASE COMMUNITY development database, the 186 development SQLs cover only 28 of the 106 enumerated simple schema paths, corresponding to 26.4% overall coverage. Coverage

decreases systematically with path depth: the development set covers 100% of length-1 paths, but only 42.9% of length-2 paths and 15.8% of length-3 paths, while covering none of the 30 length-4 paths. Thus, despite containing nearly two hundred development SQLs, the dataset leaves more than three quarters of the schema-path universe uncovered, with complete absence of direct supervision for the deepest four-edge relational structures.

C.6 BIRD Development Set: Toxicology

In the BIRD TOXICOLOGY development database, the 145 development SQLs cover 7 of the 12 enumerated simple schema paths, corresponding to 58.3% overall coverage. More importantly, coverage decreases with path depth: the development set covers 100% of length-1 paths and 75% of length-2 paths, but covers none of the four length-3 paths. Thus, although every individual foreign-key relationship is represented and most two-edge structures are exercised, the development set provides no direct supervision for any complete four-table schema chain, leaving the deepest relational structures entirely uncovered.

D DETAILED FORMULATION AND ALGORITHMS FOR SPLICE

This appendix provides the formal definitions, detailed algorithms, realization constraints, correctness properties, and complexity analysis underlying SPLICE.

D.1 Formal Schema and Question Definitions

For each table $T \in \mathcal{V}$, let $\text{cols}(T)$ denote its set of columns. For a set of tables $\mathcal{T} \subseteq \mathcal{V}$, we write its qualified columns as

$$\text{cols}(\mathcal{T}) = \bigcup_{T \in \mathcal{T}} \{T.c : c \in \text{cols}(T)\}, \quad (1)$$

i.e. every column of every table in $\mathcal{T}$, tagged by its table.

We represent an NL question by the tuple

$$q = (x, \tau(q), \chi(q), \phi(q), a(q)), \quad (2)$$

where x is the natural-language text itself, $\tau(q)$ is the set of tables it involves, $\chi(q)$ the set of columns it references, $\phi(q)$ its set of filter (predicate) patterns, and $a(q)$ its aggregation structure. These are precisely the four aspects the semantic profile constrains during realization.

For a path $P = (T_0, \dots, T_k)$ and a sub-path $P[i : j]$, we write $\text{Indices}(P[i : j]) = \{i, i + 1, \dots, j\}$ for the positions it occupies on the path. Finally, $\Pi(S)$ is the set of all simple paths in S , where a path and its reverse are treated as the same undirected path when enumerating schema coverage.

D.2 Detailed Covering-Family Algorithm

Algorithm 1 restates the greedy construction with the uncovered-position set U made explicit. It is exactly the construction used by SPLICE: it sorts the verified sub-paths from longest to shortest and greedily absorbs any sub-path that still covers an uncovered position.

This construction guarantees the coverage condition of Eq. (??) but not the contiguity condition of Eq. (??): because a sub-path is admitted whenever it intersects the still-uncovered set U (not

Algorithm 1 FindCoveringFamily: Greedy Longest-First Index Cover

Require: Path $P = (T_0, \dots, T_k)$; knowledge base $\mathcal{K}$

Ensure: Covering family $\mathcal{F}$ or $\emptyset$

```

1:  $\mathcal{S} \leftarrow \text{Sub}_{\mathcal{K}}(P)$ 
2: Sort  $\mathcal{S}$  by decreasing sub-path length; break ties by increasing
   start index
3:  $U \leftarrow \{0, 1, \dots, k\}$ 
4:  $\mathcal{F} \leftarrow []$ 
5: for all  $sp = P[i : j] \in \mathcal{S}$  in sorted order do
6:   if  $\text{Indices}(sp) \cap U \neq \emptyset$  then
7:      $\mathcal{F}.append(sp)$ 
8:      $U \leftarrow U \setminus \text{Indices}(sp)$ 
9:   end if
10:  if  $U = \emptyset$  then
11:    break
12:  end if
13: end for
14: if  $U \neq \emptyset$  then
15:   return  $\emptyset$ 
16: end if
17: Sort  $\mathcal{F}$  by starting index
18: return  $\mathcal{F}$ 

```

only when it touches the current frontier), two members that are consecutive by start index may satisfy coverage while sharing no table position. In that case the semantic extension operator is undefined between them, COMPOSESEMANTIC fails, and SPLICE uses the schema-only fallback. The procedure should therefore not be described as computing a minimum-cardinality set cover; it is a greedy longest-first path cover.

D.3 Semantic Profile Construction

For each supported table set $\mathcal{T}$, the three profile components are obtained by pooling the corresponding footprints of its verified examples:

$$C(\mathcal{T}) = \bigcup_{q \in \mathcal{K}(\mathcal{T})} \chi(q) \quad (\text{columns used}), \quad (3)$$

$$\Phi(\mathcal{T}) = \bigcup_{q \in \mathcal{K}(\mathcal{T})} \phi(q) \quad (\text{filter patterns used}), \quad (4)$$

$$A(\mathcal{T}) = \bigcup_{q \in \mathcal{K}(\mathcal{T})} a(q) \quad (\text{aggregation patterns used}), \quad (5)$$

so that $\text{Sem}(\mathcal{T}, \mathcal{K}) = (\mathcal{T}, C(\mathcal{T}), \Phi(\mathcal{T}), A(\mathcal{T}))$.

For two profiles $Z_1 = (\mathcal{T}_1, C_1, \Phi_1, A_1)$ and $Z_2 = (\mathcal{T}_2, C_2, \Phi_2, A_2)$ with $\mathcal{T}_1 \cap \mathcal{T}_2 \neq \emptyset$, the extension operator merges them component-wise by union:

$$\text{Ext}(Z_1, Z_2) = (\mathcal{T}_1 \cup \mathcal{T}_2, C_1 \cup C_2 \cup J(\mathcal{T}_1, \mathcal{T}_2), \Phi_1 \cup \Phi_2, A_1 \cup A_2), \quad (6)$$

where the aggregation component is a plain set union: every aggregation pattern observed in either sub-path remains available to the realizer. The join-key set added to the columns is

$$J(\mathcal{T}_1, \mathcal{T}_2) = \{T.c : T \in \mathcal{T}_1 \cap \mathcal{T}_2, c \text{ is a join key of } T\}, \quad (7)$$

Algorithm 2 ComposeSemantic: Iterative Semantic Profile Construction

Require: Covering family $\mathcal{F}$; knowledge base $\mathcal{K}$; schema $\mathcal{S}$

Ensure: Composed semantic profile Z_P , or failure if $\mathcal{F}$ is non-contiguous

```

1: Sort  $\mathcal{F}$  by starting index
2:  $Z \leftarrow \text{Sem}(\text{Tables}(sp_1), \mathcal{K})$ 
3: for  $\ell = 2$  to  $|\mathcal{F}|$  do
4:   if  $Z.\mathcal{T} \cap \text{Tables}(sp_\ell) = \emptyset$  then
5:     fail  $\triangleright$  Contiguity violated; caller uses SchemaProfile
6:   end if
7:    $Z \leftarrow \text{Ext}(Z, \text{Sem}(\text{Tables}(sp_\ell), \mathcal{K}))$ 
8: end for
9: return  $Z$ 

```

i.e. the key columns of the shared table(s) through which the two sub-paths are joined.

The schema-only fallback, used when no composable covering family exists, is

$$\text{SchemaProfile}(P, \mathcal{S}) = (\text{Tables}(P), \text{cols}(\text{Tables}(P)), \emptyset, A_{\text{std}}). \quad (8)$$

Reading the four components in order: the table footprint is the full table set of the path; the columns are *all* columns of *all* those tables; the filter-pattern set is *empty* (no verified filters are asserted); and the aggregation set is just the standard vocabulary A_{std} . This is strictly weaker grounding than a composed profile—it relies on schema structure rather than on verified query behavior—and is therefore recorded as a distinct, lower-confidence generation condition.

Given an ordered covering family $\mathcal{F} = [sp_1, \dots, sp_m]$, the detailed composition procedure is Algorithm 2. It sorts the family by starting index, seeds the running profile with the first sub-path, and extends it one sub-path at a time, failing if any adjacent pair turns out not to share a table (the contiguity check).

D.4 Correctness Properties

PROPOSITION D.1 (COMPLETE INDEX COVERAGE). *If Algorithm 1 returns a non-empty family $\mathcal{F} = (F_1, \dots, F_m)$, then*

$$\bigcup_{\ell=1}^m \text{Indices}(F_\ell) = \{0, \dots, k\}. \quad (9)$$

PROOF. The algorithm maintains the set U of path positions not yet covered, initialized to $\{0, \dots, k\}$. Each admitted sub-path only removes indices from U , and a sub-path is admitted whenever it intersects U . The loop terminates either with $U = \emptyset$ or after exhausting $\mathcal{S}$; in the latter case the algorithm returns $\emptyset$. Hence a non-empty return implies $U = \emptyset$, i.e. the removed index sets together cover $\{0, \dots, k\}$. $\square$

PROPOSITION D.2 (COVERAGE DOES NOT IMPLY COMPOSABILITY). *A non-empty family returned by Algorithm 1 satisfies the coverage condition of Eq. (??) but need not satisfy the contiguity condition of Eq. (??).*

PROOF. A sub-path is admitted whenever its index set intersects the current uncovered set U , regardless of whether it touches the

previously admitted sub-paths. Consider a path of length $k = 4$ with $P[2 : 4], P[0 : 1] \in \text{Sub}_{\mathcal{K}}(P)$ but $P[1 : 2] \notin \text{Sub}_{\mathcal{K}}(P)$. Sorted longest-first, the algorithm admits $P[2 : 4]$ (removing $\{2, 3, 4\}$) and then $P[0 : 1]$ (removing $\{0, 1\}$), yielding coverage of $\{0, \dots, 4\}$. Ordered by start index the family is $(P[0 : 1], P[2 : 4])$, for which $i_2 = 2 > j_1 = 1$, so contiguity fails. Consequently $\text{Tables}(P[0 : 1]) \cap \text{Tables}(P[2 : 4]) = \emptyset$ and Ext is undefined between them. $\square$

REMARK 1 (COMPOSITION FALLBACK). *When the returned family violates contiguity, COMPOSESEMANTIC fails (Algorithm 2) and SPLICE uses SchemaProfile(P, S) of Eq. (8). A family is composable precisely when COMPOSESEMANTIC succeeds, i.e. when every consecutive pair (by start index) shares at least one table.*

PROPOSITION D.3 (LOCAL-TO-GLOBAL SEMANTIC COVERAGE). *If a target path P is directly uncovered but Algorithm 1 returns a contiguous non-empty family $\mathcal{F}$, then SPLICE constructs a composed semantic profile whose table footprint is exactly $\text{Tables}(P)$, using only semantics retrieved from verified sub-paths and schema-defined join keys.*

PROOF. Every member of $\mathcal{F}$ belongs to $\text{Sub}_{\mathcal{K}}(P)$ and therefore has a verified semantic profile. By complete index coverage, the union of their table sets equals $\text{Tables}(P)$. Contiguity makes each consecutive extension defined, so the recursive application of Ext unions the table and column footprints of consecutive profiles and explicitly adds the join-key columns at their interfaces. Hence the resulting profile has table footprint $\text{Tables}(P)$ and carries semantic evidence spanning the complete target path. (When $\mathcal{F}$ is non-contiguous, SPLICE instead returns SchemaProfile(P, S), whose footprint is also $\text{Tables}(P)$ but whose evidence is schema-only.) $\square$

D.5 Constrained NL Generation

Given a profile $Z_P = (\mathcal{T}_P, C_P, \Phi_P, A_P)$, the generation function can be read as a pipeline that, in turn, selects columns, selects filters, selects an aggregation, and produces an NL realization:

$$N(Z_P, n_{\min}) = I \circ \mathcal{A} \circ \mathcal{F} \circ C(Z_P, n_{\min}), \quad (10)$$

where C selects columns, $\mathcal{F}$ selects filters, $\mathcal{A}$ selects aggregations, and I produces the natural-language text (the symbol $\circ$ denotes function composition, applied right to left).

For each generated question q , the constraints are

$$\tau(q) = \mathcal{T}_P, \quad \chi(q) \subseteq C_P, \quad (11)$$

$$\phi(q) \subseteq \Phi_P, \quad a(q) \in A_{\text{std}} \cup A_P. \quad (12)$$

Columns and filters are restricted to the composed profile, whereas the aggregation stage $\mathcal{A}$ offers the fixed standard vocabulary A_{std} together with the observed patterns A_P , independently of whether Z_P came from composition or from the schema-only fallback; the realizer is instructed to vary the aggregation across the $n_{\min}$ questions.

The source implementation additionally supplies few-shot NL-SQL examples drawn from the covering family to ground the LLM realization. The prompt can therefore be summarized as Prompt = $(P, Z_P, \mathcal{E}_P, \mathcal{I}_P)$, where $\mathcal{E}_P$ holds the representative verified examples (empty under the schema-only fallback) and $\mathcal{I}_P$ holds the generation constraints.

D.6 Detailed SPLICE Algorithm

Algorithm 3 SPLICE: Detailed End-to-End Procedure

Require: $S, \mathcal{K}$, window $[t_{\min}, t_{\max}]$, path budget $B_P, n_{\min}$

Ensure: Q_{SPLICE}

```

1:  $\mathcal{P}_F \leftarrow$  deepest-first, round-robin-across-database ordering of
    $\Pi_{[t_{\min}, t_{\max}]}(S)$ , truncated to  $B_P$ 
2:  $Q_{\text{SPLICE}} \leftarrow \emptyset$ 
3: for  $P \in \mathcal{P}_F$  do
4:    $\mathcal{T}_P \leftarrow \text{Tables}(P)$ 
5:    $\mathcal{F} \leftarrow \text{FINDCOVERINGFAMILY}(P, \mathcal{K})$ 
6:   if  $\mathcal{F} = \emptyset$  then
7:      $Z_P \leftarrow \text{SCHEMAPROFILE}(P, S)$ 
8:   else
9:     try  $Z_P \leftarrow \text{COMPOSESEMANTIC}(\mathcal{F}, \mathcal{K}, S)$ 
10:    except composition failure:  $Z_P \leftarrow$ 
        SCHEMAPROFILE( $P, S$ );  $\mathcal{F} \leftarrow \emptyset$ 
11:   end if
12:    $\mathcal{G}_P \leftarrow \mathcal{N}(P, Z_P, n_{\min})$ 
13:   for  $q \in \mathcal{G}_P$  do
14:     if Valid( $q, P, Z_P$ ) then
15:       Attach provenance  $\text{Prov}(q) = (P, \mathcal{F}, Z_P)$ 
16:        $Q_{\text{SPLICE}} \leftarrow Q_{\text{SPLICE}} \cup \{q\}$ 
17:     end if
18:   end for
19: end for
20: return  $Q_{\text{SPLICE}}$ 

```

D.7 Complexity Analysis

For a path of length k , the number of contiguous sub-paths is $\binom{k+1}{2} = O(k^2)$. Assuming hash-indexed access to $\text{supp}(\mathcal{K})$, identifying the semantically covered sub-paths therefore requires expected $O(k^2)$ lookup time.

Let $m = |\text{Sub}_{\mathcal{K}}(P)|$ be the number of covered sub-paths. Sorting them by length requires $O(m \log m) \subseteq O(k^2 \log k)$ time. The greedy scan makes a single pass over the m sorted candidates, each step performing an index-set intersection/removal in $O(k)$, so the scan costs $O(mk)$ in the worst case. The conservative complexity of the covering stage is thus

$$O(k^2 \log k + mk). \quad (13)$$

If the selected family has r members and the corresponding semantic profiles contain at most C, Φ , and A column, filter, and aggregation elements respectively, iterative composition requires $O(r(C + \Phi + A))$ time under hash-based set-union operations.

For Q generated candidates, structural validation requires $O(Qk)$ time, and pairwise semantic deduplication, if performed, costs $O(Q^2)$ in the worst case. Consequently, excluding the external cost of LLM inference, the per-path deterministic complexity is bounded by

$$O(k^2 \log k + mk + r(C + \Phi + A) + Qk + Q^2). \quad (14)$$

For a target set $\mathcal{P}_F$, the total deterministic complexity is the sum of the per-path bounds:

$$O\left(\sum_{P \in \mathcal{P}_F} [k_P^2 \log k_P + m_P k_P + r_P(C_P + \Phi_P + A_P) + Q_P k_P + Q_P^2]\right). \quad (15)$$

LLM realization cost is external to this algorithmic bound and depends on the number of generation calls and the token length of the supplied schema, semantic profile, and demonstrations.

D.8 Provenance

For every generated instance q , SPLICE can retain its *provenance*

$$\text{Prov}(q) = (P, \mathcal{F}, Z_P), \quad (16)$$

where P is the target path, $\mathcal{F}$ identifies the verified sub-paths that contributed semantic evidence (empty under the schema-only fallback), and Z_P is the profile used for generation. This makes it possible to trace an NL instance back to the structural region that motivated it and to the verified semantic evidence used for its realization—and, in particular, to distinguish composed instances from lower-confidence schema-only ones.

D.9 Summary of the SPLICE Mechanism

The principal distinction is therefore between a *directly covered* region and a *composable uncovered* region. SPLICE specifically exploits the latter: the complete target table set need not occur in the verified knowledge base provided that its contiguous portions admit a composable covering family; and when they do not, SPLICE falls back to a schema-only profile so that even the hardest paths still yield evaluation instances.